CSD 330 – Security Analytics

Homework #2

Theoretical Exercises

1. Why would URL investigation help in security/privacy? Give at least three different applications from security/privacy in which it could be useful.

Answer:

A Uniform Resource Locator, colloquially termed a web address, is a reference to a web r esource that specifies its location on a computer network and a mechanism for retrieving it. A URL is a specific type of Uniform Resource Identifier, although many people use the two terms interchangeably. It is basically a set of directions for a web browser to reach a webpage/file.

By examining the URL we can try to determine the destination of the URL without actuall y clicking on the URL. We can then decide to proceed clicking on it if it comes from a tr usted source and leads to a trusted destination. If anything suspicious or a bunch of stran ge characters are found in URL investigation we should report the URL or use a link scan ner to validate it and definitely not click it, as it may contain directions to some malware or other malicious files/sites.

We use URL Investigation to prevent users from accessing websites that provide content t hat is objectionable, potentially harmful, or not work-
related. Applications of URL investigation –

a) Detection of Malicious Objects – There are many anti-
virus software such as Kaspersky which us URL investigation to classify URLs into different categories such as –

- Good – URL not malicious.
- Dangerous – Malicious objects related to the URL.
- Adware and Other – Objects related to the URL, which are classified as NOT-avirus.
- Not categorised – No information about the URL is available

b) Detection of Phishing Website and Emails –
Emails that claim to be from well know banks and organisation and have links for the us er to “verify their information” are often phishing mails. We should inspect the link caref

ully and check it legitimacy before clicking on it. Multiple softwares, such as PhishTank, use URL investigation to check for phishing markers.

c) Prevention of Credential Based Attacks –
URL investigation is used to prevent credential based attacks. Software can identify and prevent attempts to steal credentials by stopping users from submitting password-based corporate credentials to unknown sites

2. How could a text document be represented as a market-basket type of transaction? Give at least two different ways.

Answer:

1. In our approach, we model text documents as transactions where items are words or phrases from the document as well as the categories to which the document belongs, as described above. A data cleaning phase is required to weed out those words that are of no interest in building the associative classifier. We consider stopwording and term pruning as well as the transformation of documents into transactions as a pre-processing phase. Stopword removal and term pruning is done according to the TF/IDF values and a given list of stopwords. We have opted to selectively turn on and off stopwording depending upon the data set to categorize. It is only after the terms are selected from the cleansed documents that the transactions are formed. The subsequent phase consists of discovering association rules from the set of cleansed transactions.

2. We can process the text document using stemming and other word normalization methods to create an index. We can then use this index and find the cosine similarity score between different documents and therefore this score can act as the magnitude of association between the two documents. We can then suggest documents which have high similarity score when a user opens/reads a text document.

3. We assume that the text document contains the list of items bought in a sequence in which it is present in the document i.e " Car House" implies that car was bought before the house and both of them are in the same shopping list. We will have many such lists and by analysing and processing them we can acquire user buying tendencies and associations between different items. We can then use these patterns and associations to suggest new items to the user.

3. How could URLs be clustered? Give at least two different distance functions for URLs. How many of the distance functions you listed satisfy the definition of a metric? Prove or disprove for one distance function whether it is a metric or not.

Answer:

URLs can be clustered using distance-based approaches. We can calculate different distance functions for each URL. We select r

andom URLs equal to the number of clusters required and call them header URLs. For the rest we calculate the distance between a URL and each of the header URLs. We then the n associate/cluster the URL with the header URL which is the closest to it (Shortest distan ce).

Distance Functions for URLs –

• Levenshtein Distance –
Every editing operation such as insert, remove and replace have unitary cost. Copy opera tion has zero cost. This calculates the distance between pairs of strings that is at most e qual to the length of the longer string. It basically is the distance i.e the number of unar y operations required to convert one to URL to another. The number of operations the re quired distance.

• Cosine Similarity –
We first take all the words present in the URLs and make an index for it. We also docu ment the frequency of the words present in the URL. We then take the logarithmic of the frequency, of each word separately, and add 1 to it. We then normalise the weight for e ach word by dividing it by the root mean square. By this we will get weights for each wo rd in each URL and we then multiple the weights of the common words between the URL s and sum them to get the Cosine similarity score.

• Jaro Distance –
This distance function considers the number and the order of common characters betwee n two URLs. If m is the number of matching characters and t is half of the number of tr anspositions required then Jarao Distance(u,v) = 1 – 1/3 (m/u + m/v + (m-t)/m) = 1 when m=0; u and v are the two URLs.

Jaro distance and Levenshtein distance are both metrics. A metric is a function that defin e a distance between each pair of elements of a set. The levenshtein distance will also pr ovide the distance between any two URLs. No integration or dot product is required.

Levenshtein Distance is always >=0

ii. It is symmetric.

iii. If Levenshtein distance = 0 then the URLs are equal.

iv. Satisfies Triangle inequality.

Therefore, Levenshtein distance is a metric

4. Repeat Questions 1 and 3 for a TCP/IP Packet.

Answer:

Part 1

The header contains information about the packet such as the version of protocol being used, source and destination addresses, checksum, number of packets used to send the entire data, and Time to Live data. TCP/IP packets contain segments of data to be transferred along with where the data is to be sent. Each packet consists of a packet header and data. These are all vital information to check for the validity of packet being sent over a network. We use packet investigation to filter packets from suspicious senders. We can investigate the header contents of the packet to ensure that the data is in correct format, to check for malicious code and for eavesdropping attacks.

Application of TCP/IP packet investigation are –

a) Detecting malicious code –
Using packet investigation we can reveal suspicious header contents and raise an alert to the system administrator. We can also check for faults in the chain of custody of a given packet.

b) Detecting Eavesdropping attacks –
attacks such as Man in the Middle Attack can be identified through packet investigation. The Time to Live data will definitely be affected if there is some third party attacker hijacking the data packet in between transmission from source to destination.

c) Packet investigation enables advanced network management, user service and security functions as well as internet Data Mining through Deep Packet Inspection. We can administer and manage various computer networks and provide fault analysis and performance management of the network and filter out packets from suspicious senders.

Part 2

A. Packets requiring the same number of jumps/hops will be clustered together. Distance vector routing(DVR) algorithm can be used to find the distance between two packets. Each hop will have some weight associated to it. This weight can we calculated using the Bell-Ford Algorithm. Let dx(y) be the cost of the least-
cost path from node x to node y. The least costs are related by Bellman-
Ford equation, dx(y) = minv(c(x.v) + dv(y)) Where the minv is the equation taken for all x neighbors.

B. Clustering can also be done based on the data vector that each information packet has. The same distance functions mention above(for URLs) can be used when considering the

data vectors. Jaro, Levensthein and DVR distance are metric distances. A metric is a functi on that define a distance between each pair of elements of a set. The levenshtein distanc e will also provide the distance between any two URLs. No integration or dot product is r equired.

Levenshtein Distance is always >=0

It is symmetric.

If Levenshtein distance = 0 then the URLs are equal.

Satisfies Triangle inequality.

Therefore, Levenshtein distance is a metric.

5. Reading exercise: study conditional independence of random variables. For the Iris flower data set from UCI repository, first discretize independently the range of values for each attribute into three categories (call them small, medium and large) representing intervals of equal size and th en check whether petal length is conditionally independent of the sepal width when given the se pal length.

Answer:

Please refer to the attached photographs of my solution and the attached Excel where I have an alysed the dataset. *(HW1 – Part A – Q5.xlsx)*

We know,

For two conditionally independent events,

$$P(A \cap B / C) = P(A/C) \cdot P(B/C)$$

Now,

If $P(B) > 0$

by conditioning on C, we get

$$P(A/B, C) = \frac{P(A \cap B/C)}{P(B/C)}$$

If they are conditionally independent,

$$= \frac{P(A/C) \cdot \cancel{P(B/C)}}{\cancel{P(B/C)}}$$

$$= \underline{\underline{P(A/C)}}$$

Hence, we need to check if

$$\frac{P(A \cap B/C)}{P(B/C)} = P(A/C)$$

Let us start with

$P(A)$ = Prob. that sepal width is SMALL

$P(B)$ = Prob. that petal length is SMALL

$P(C)$ = Prob. that sepal length is SMALL

* Refer to attached Excel file here.

We see

$P(A) = 57/150$

$P(B) = 47/150$

$P(C) = 50/150$

$$P(A \cap B/C) = \frac{P(A \cap B \cap C)}{P(C)} = \frac{12/150}{50/150} = \frac{6}{25}$$

or $\underline{\underline{24\%}}$

$$P(B/C) = \frac{P(B \cap C)}{P(C)} = \frac{13/150}{50/150} = \frac{13}{50}$$

or $\underline{\underline{26\%}}$

$$P(A/C) = \frac{P(A \cap C)}{P(C)} = \frac{40/150}{50/150}$$

$$= \frac{40}{50} \quad \text{or} \quad \underline{\underline{80\%}}$$

Now,

$$LHS = \frac{P(A \cap B/C)}{P(B/C)} = \frac{24}{26} = \frac{12}{13} \approx \underline{\underline{92\%}}$$

$$RHS = \underline{\underline{80\%}}$$

$$\therefore LHS \neq RHS$$

Hence, they are not conditionally independent

(Just one counter-example is sufficient to show the same)

Practical Exercises

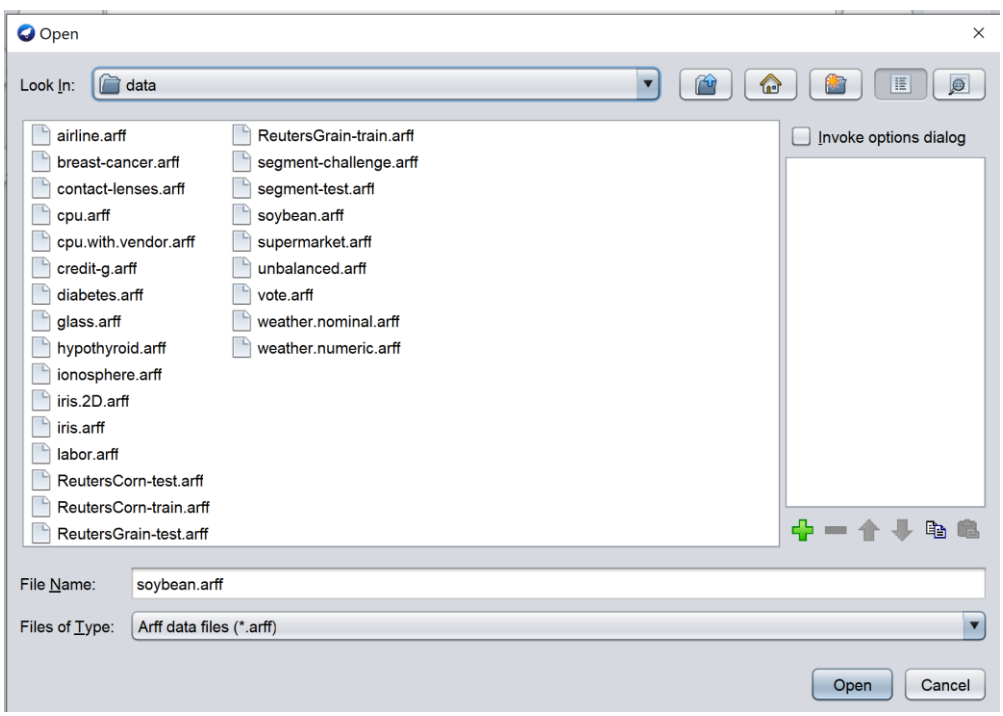
1. Finding Datasets: (a) What kind of file format does Weka use? (b) Are there any such datasets on your computer? (c) Find at least one and load it into Weka (Go to / and run find command) (d) Record the dataset and your observations (RYO) (e) Are any of the attributes correlated in the dataset you loaded.

Answer:

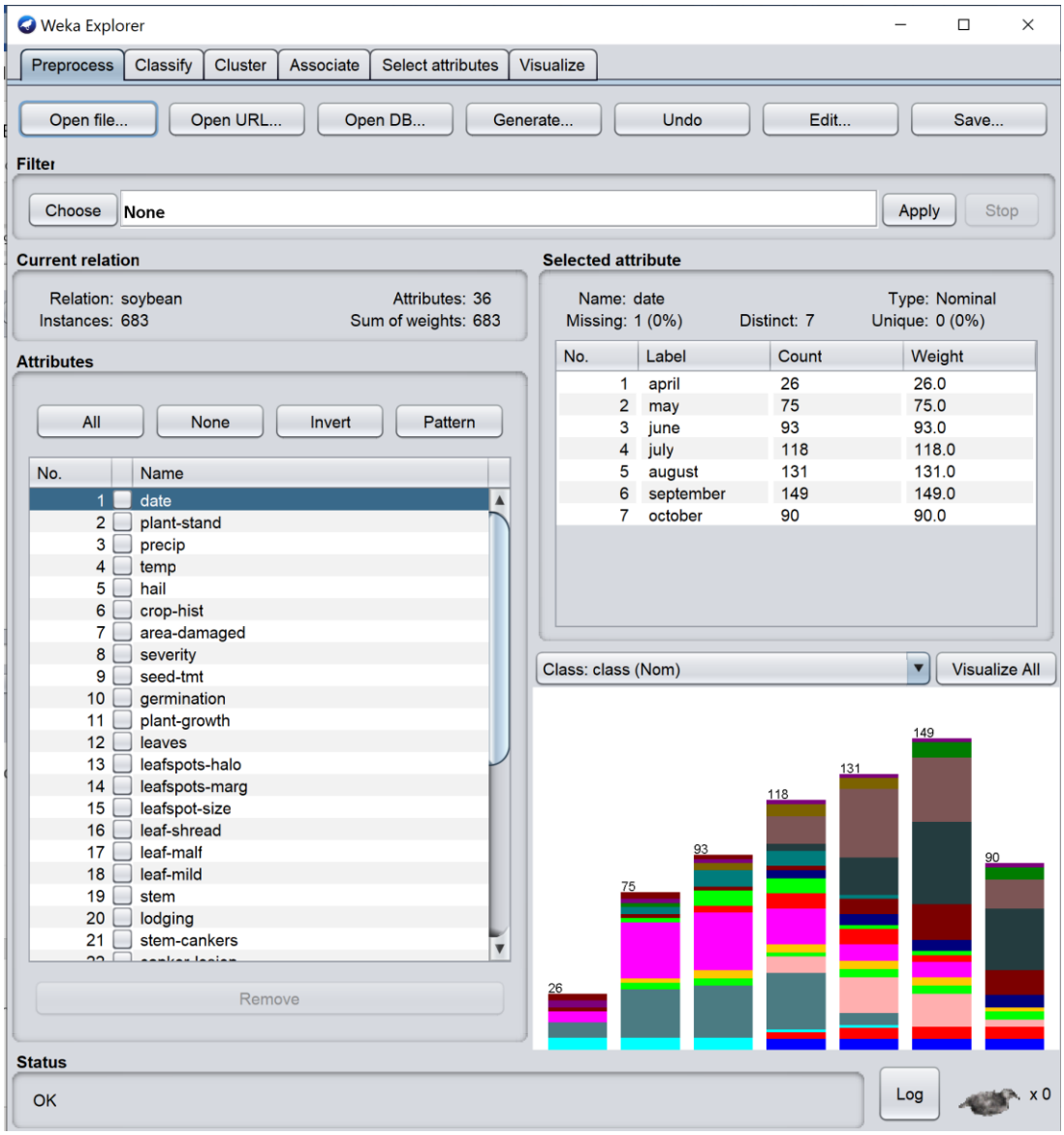
a) Weka uses .arff file format. **arff** stands for attribute relation file format. It is an ASCII text file that describes a list of instances sharing a set of attributes. It is an extension of CSV. Weka also supports the following file formats:

- .arff.gz
- .names
- .data
- .csv
- .json
- .json.gz
- .libsvm
- .m
- .dat
- .bsi
- .xrff
- .xrff.gz

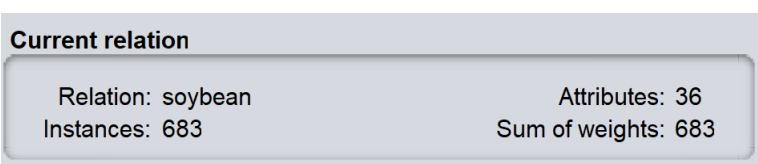
b) No, my computer did not have any such datasets. However, some datasets come preins talled with Weka and can be found at *C:\Program Files\Weka-3-8-4\data.*



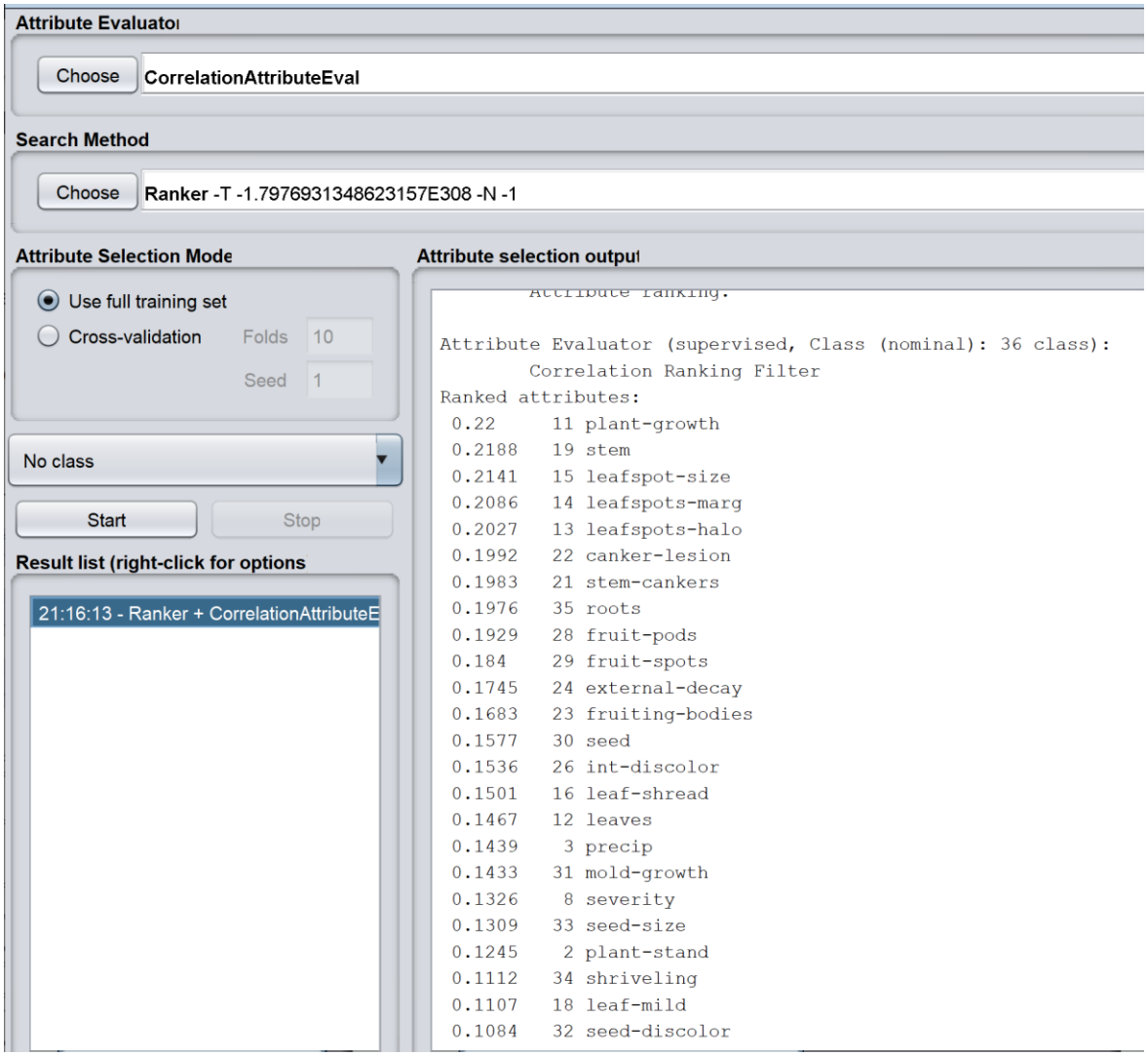
c) For my observations, I used the Weka dataset called *soybean.arff*



d) The data set contains 36 attributes. Each attribute has about 10-15 classes. Most of them are nominal. There are 683 instances. We can see bar graphs created for each attribute. There are various functions present to filter, cluster, classify, and associate the dataset.



e) Please refer to the screenshot.



2. (a) Get security datasets from the UCI Machine Learning repository. Lets use the Phishing web site dataset (preferably one which has 1353 instances, or 11000 instances) (b) Use the Explorer t ab in Weka (c) Understand the features from the documentation (d) How many instances are the re in the dataset??? RYO (e) Load the dataset into Weka (f) Visualize and RYO (g) Find Informati on Gain

<u>Answer:</u>

a)

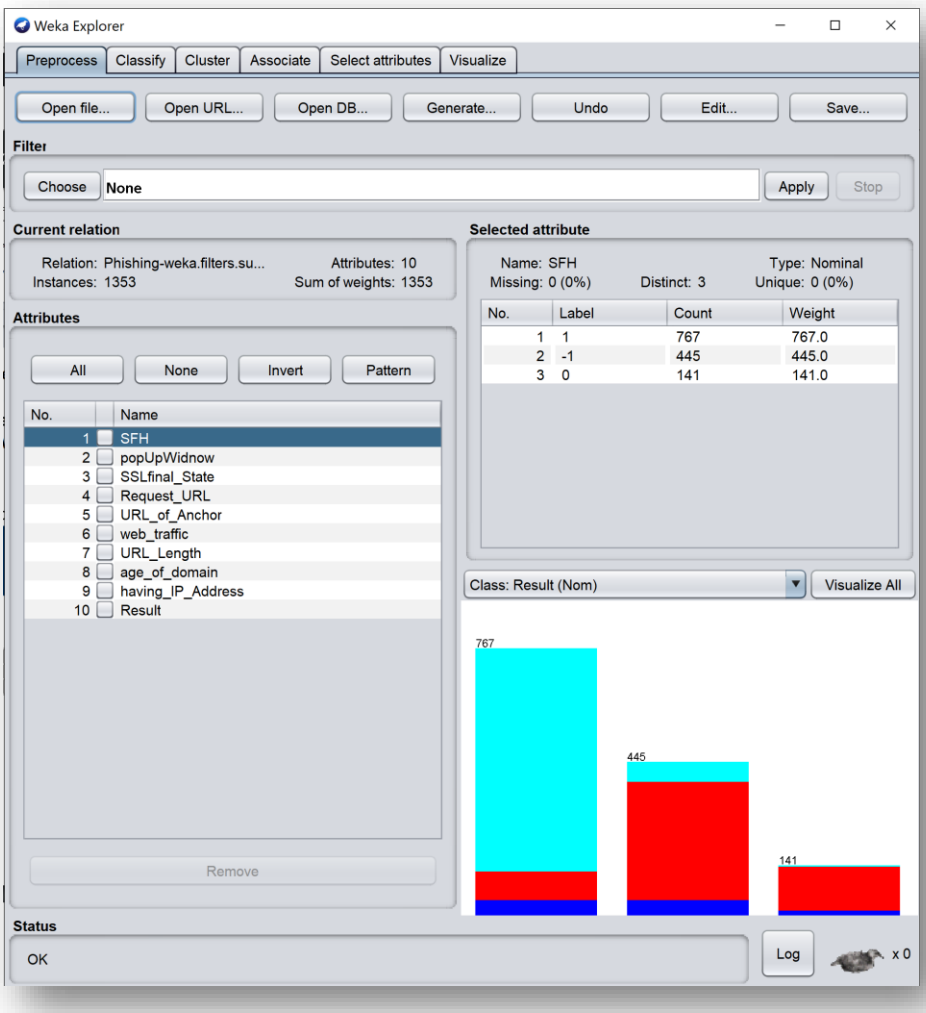
Website Phishing Data Set

Download: Data Folder, Data Set Description

Abstract:

Data Set Characteristics:	Multivariate	**Number of Instances:**	1353	**Area:**	Computer
Attribute Characteristics:	Integer	**Number of Attributes:**	10	**Date Donated**	2016-11-02
Associated Tasks:	Classification	**Missing Values?**	N/A	**Number of Web Hits:**	73797

b) Weka Explorer Tab

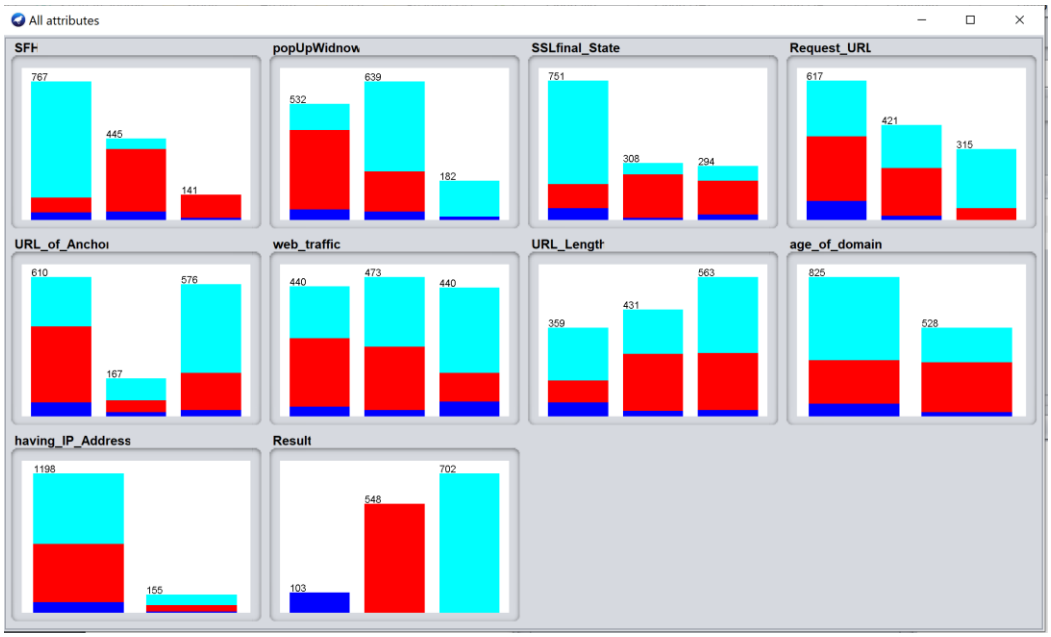


c) **Data Set Information:**

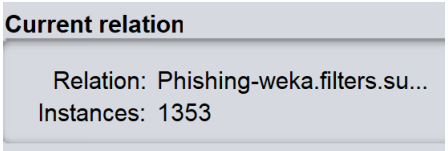
The phishing problem is considered a vital issue in the security industry especially e-banking and e-commerce taking the number of online transactions involving payments. We have identified different features related to legitimate and phishy websites and collected 1353 different websites from difference sources. Phishing websites were collected from Phishtank data archive (www.phishtank.com), which is a free community site where users can submit, verify, track and share phishing data. The legitimate websites were collected from Yahoo and starting point directories using a web script developed in PHP. The PHP script was plugged with a browser and we collected 548 legitimate websites out of 1353 websites. There is 702 phishing URLs, and 103 suspicious URLs. When a website is considered SUSPICIOUS that means it can be either phishy or legitimate, meaning the website held some legit and phishy features.

Attribute Information:

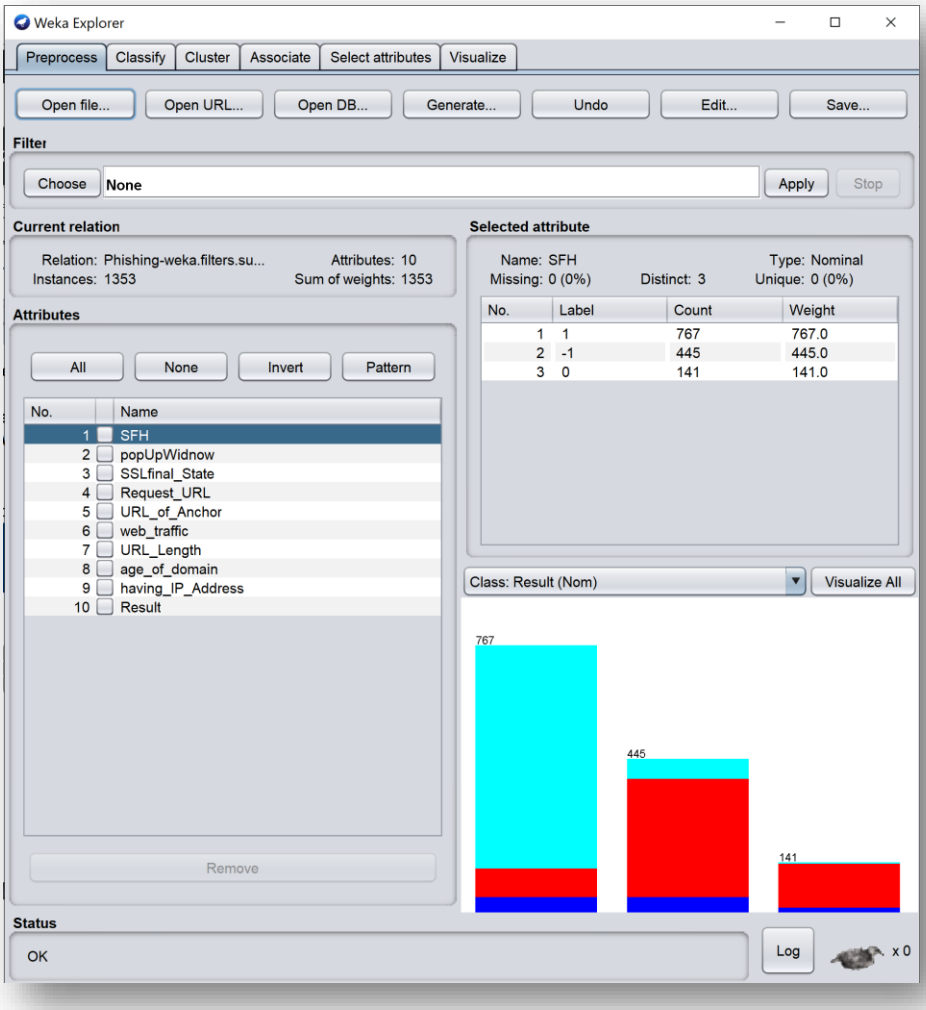
URL Anchor
Request URL
SFH
URL Length
Prefix/Suffix
IP
Sub Domain
Web traffic
Domain age
Class



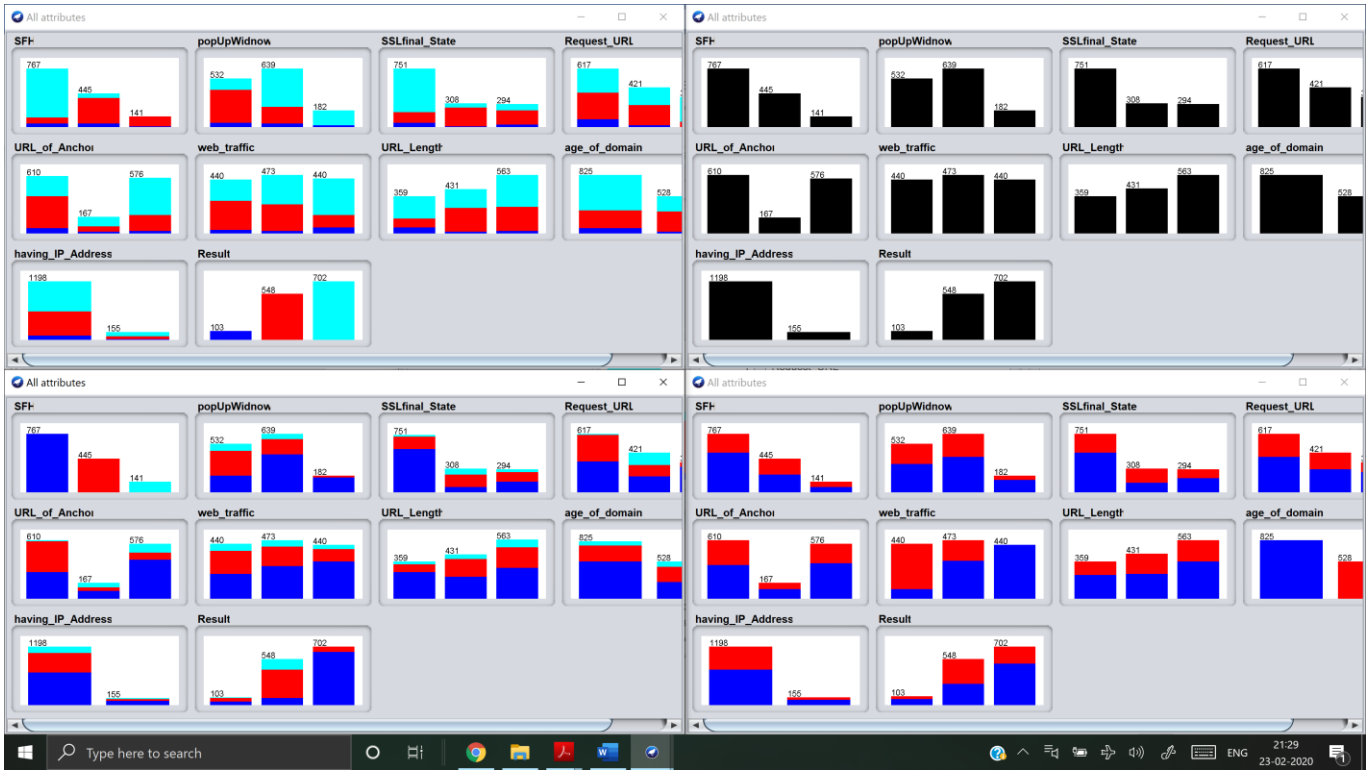
d) 1353 instances



e) Loaded into Weka



f) Some of the various possible visualisations



There are 10 attributes present. Each attribute has 3 classes {-1,0,1} except attribute "having_IP_address" which has only 2. The data is unevenly spread. All attributes are Nominal in type.

g)

```
Attribute Evaluator (supervised, Class (nominal): 10 Result):
        Information Gain Ranking Filter

Ranked attributes:
 0.45494    1 SFH
 0.22842    2 popUpWidnow
 0.22741    3 SSLfinal_State
 0.11811    4 Request_URL
 0.06906    5 URL_of_Anchor
 0.05723    6 web_traffic
 0.04916    7 URL_Length
 0.04312    8 age_of_domain
 0.00271    9 having_IP_Address

Selected attributes: 1,2,3,4,5,6,7,8,9 : 9
```

3. (a) Decision Trees (What does Weka call its decision tree classifiers and why?) (b) Record the confusion matrix and the weighted Recall Precision F-score results you get (c) Examine the model how deep is the decision tree? Can you generate rules from the decision tree? (d) Now select the attribute with the highest information gain and just do a decision tree using this attribute? What are the results? Compare with the previous step with all attributes?

Answer:

a) There are eight different decision tree classifiers in Weka:

1. J48
2. DecisionStump
3. RandomTree
4. RandomForest
5. REPTree
6. M5P
7. LMT
8. HoeffdingTree

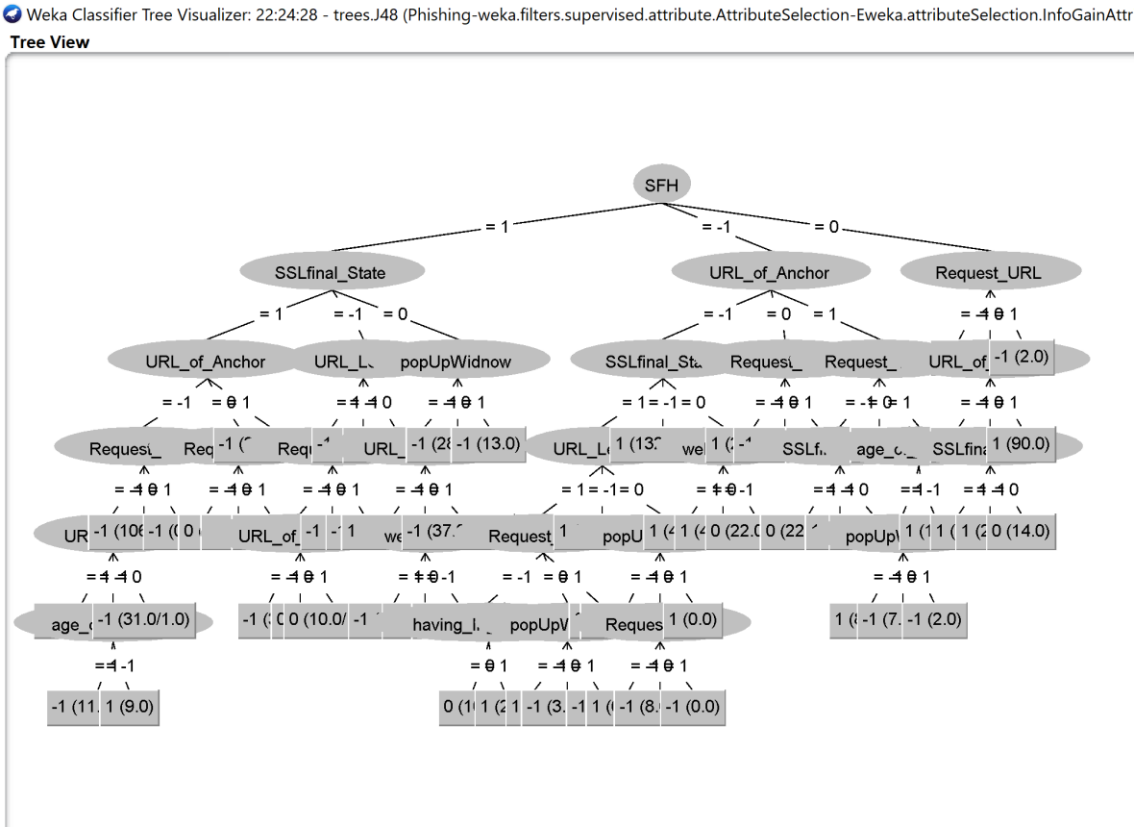
However, the most commonly used one is **J48**, which is called so because it uses C4.5 (J48) for generating the trees.

b)

Classifier output

```
Size of the tree :      88


Time taken to build model: 0.02 seconds

=== Stratified cross-validation ===
=== Summary ===

Correctly Classified Instances        1228               90.7613 %
Incorrectly Classified Instances       125                9.2387 %
Kappa statistic                          0.8363
Mean absolute error                      0.0831
Root mean squared error                  0.2213
Relative absolute error                 22.199  %
Root relative squared error             51.1815 %
Total Number of Instances             1353

=== Detailed Accuracy By Class ===

                 TP Rate  FP Rate  Precision  Recall   F-Measure  MCC      ROC Area  PRC Area  Class
                 0.932    0.015    0.835      0.932    0.881      0.872    0.986     0.846     0
                 0.892    0.065    0.904      0.892    0.898      0.829    0.958     0.938     1
                 0.916    0.083    0.923      0.916    0.919      0.833    0.958     0.943     -1
Weighted Avg.    0.908    0.070    0.908      0.908    0.908      0.834    0.960     0.934

=== Confusion Matrix ===

   a   b   c   <-- classified as
  96   2   5 |   a = 0
  10 489  49 |   b = 1
   9  50 643 |   c = -1
```

c)



The decision tree is 7 levels deep.

Yes, we can generate rules through the decision tree through traversing a path from the r oot node to any leaf node.

d)

Classifier output

```
Size of the tree :      33


Time taken to build model: 0 seconds

=== Stratified cross-validation ===
=== Summary ===

Correctly Classified Instances        1197               88.4701 %
Incorrectly Classified Instances       156               11.5299 %
Kappa statistic                          0.794
Mean absolute error                      0.1315
Root mean squared error                  0.2583
Relative absolute error                 35.2432 %
Root relative squared error             59.8162 %
Total Number of Instances             1353

=== Detailed Accuracy By Class ===

                 TP Rate  FP Rate  Precision  Recall   F-Measure  MCC      ROC Area  PRC Area  Class
                 0.907    0.125    0.905      0.907    0.906      0.783    0.898     0.876     1
                 0.825    0.081    0.832      0.825    0.828      0.745    0.874     0.797     -1
                 0.950    0.007    0.937      0.950    0.944      0.937    0.977     0.902     0
Weighted Avg.    0.885    0.098    0.884      0.885    0.885      0.787    0.899     0.853

=== Confusion Matrix ===

   a   b   c   <-- classified as
 696  69   2 |   a = 1
  71 367   7 |   b = -1
   2   5 134 |   c = 0
```

All the attributes for comparison are available in the two screenshots.

4. Apriori algorithm: (a) Use a support threshold of 0.1, and find frequent itemsets (b) Then use confidence of 0.8 with the same support threshold Are there any rules? (c) Increase support to 0. 5 and find frequent itemsets (d) Is the dataset unbalanced? How do you find out? (e) Create an unbalanced dataset of ratio 1:10 using random selection (f) Run decision tree algorithm on unbal anced dataset (g) Compare the results for 2 classes with balanced versus unbalanced.

<u>Answer:</u>

a)

Associator output

```
Apriori
=======

Minimum support: 0.1 (135 instances)
Minimum metric <confidence>: 0.9
Number of cycles performed: 18

Generated sets of large itemsets:

Size of set of large itemsets L(1): 27

Size of set of large itemsets L(2): 185

Size of set of large itemsets L(3): 344

Size of set of large itemsets L(4): 213

Size of set of large itemsets L(5): 54

Size of set of large itemsets L(6): 6

Best rules found:

 1. web_traffic=-1 URL_Length=0 Result=-1 135 ==> age_of_domain=1 135    <conf:(1)> lift:(1.64) lev:(0.04) [52] conv:(52.68)
 2. SFH=1 SSLfinal_State=1 Request_URL=0 137 ==> Result=-1 135    <conf:(0.99)> lift:(1.9) lev:(0.05) [63] conv:(21.97)
 3. SSLfinal_State=-1 Request_URL=-1 139 ==> having_IP_Address=0 135    <conf:(0.97)> lift:(1.1) lev:(0.01) [11] conv:(3.18)
 4. URL_Length=1 age_of_domain=1 Result=-1 146 ==> SFH=1 135    <conf:(0.92)> lift:(1.63) lev:(0.04) [52] conv:(5.27)
 5. popUpWidnow=1 SSLfinal_State=1 147 ==> Result=-1 135    <conf:(0.92)> lift:(1.77) lev:(0.04) [58] conv:(5.44)
 6. SFH=1 Request_URL=0 having_IP_Address=0 147 ==> Result=-1 135    <conf:(0.92)> lift:(1.77) lev:(0.04) [58] conv:(5.44)
 7. URL_of_Anchor=-1 web_traffic=0 Result=1 148 ==> having_IP_Address=0 135    <conf:(0.91)> lift:(1.03) lev:(0) [3] conv:(1.21)
 8. Request_URL=1 URL_of_Anchor=1 age_of_domain=1 having_IP_Address=0 149 ==> Result=-1 135    <conf:(0.91)> lift:(1.75) lev:(0.04) [57] conv:(4.78)
```

b)

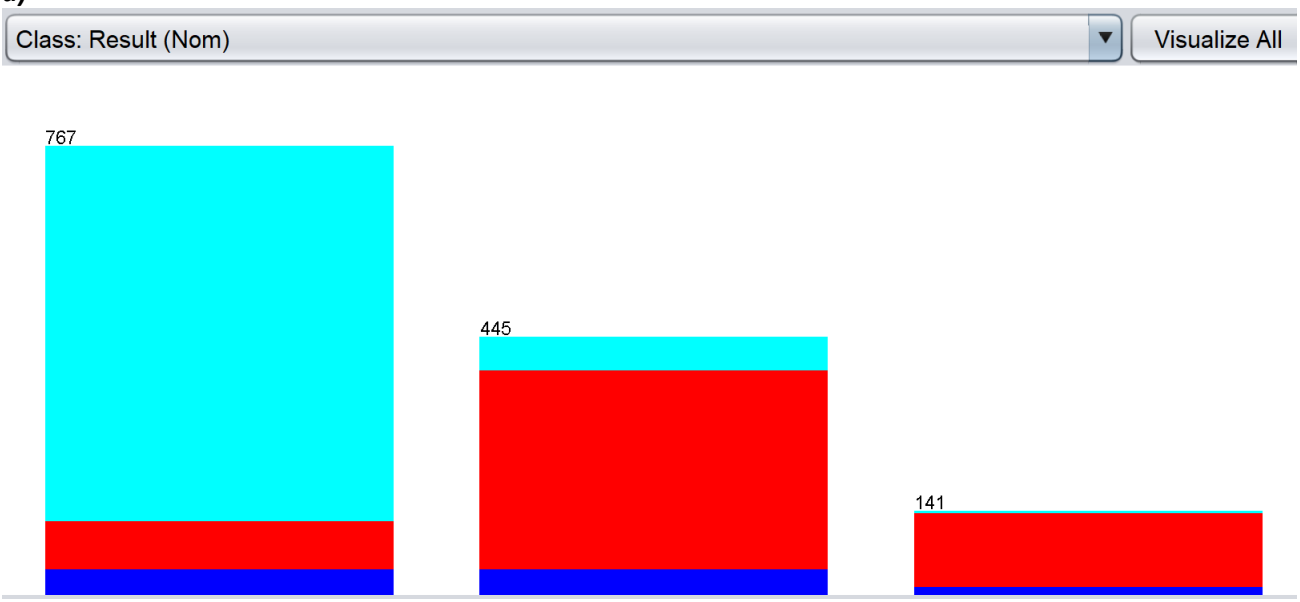
Associator output

```
Minimum support: 0.1 (135 instances)
Minimum metric <confidence>: 0.8
Number of cycles performed: 18

Generated sets of large itemsets:

Size of set of large itemsets L(1): 27

Size of set of large itemsets L(2): 185

Size of set of large itemsets L(3): 344

Size of set of large itemsets L(4): 213

Size of set of large itemsets L(5): 54

Size of set of large itemsets L(6): 6

Best rules found:

 1. web_traffic=-1 URL_Length=0 Result=-1 135 ==> age_of_domain=1 135    <conf:(1)> lift:(1.64) lev:(0.04) [52] conv:(52.68)
 2. SFH=1 SSLfinal_State=1 Request_URL=0 137 ==> Result=-1 135    <conf:(0.99)> lift:(1.9) lev:(0.05) [63] conv:(21.97)
 3. SSLfinal_State=-1 Request_URL=-1 139 ==> having_IP_Address=0 135    <conf:(0.97)> lift:(1.1) lev:(0.01) [11] conv:(3.18)
 4. URL_Length=1 age_of_domain=1 Result=-1 146 ==> SFH=1 135    <conf:(0.92)> lift:(1.63) lev:(0.04) [52] conv:(5.27)
 5. popUpWidnow=1 SSLfinal_State=1 147 ==> Result=-1 135    <conf:(0.92)> lift:(1.77) lev:(0.04) [58] conv:(5.44)
 6. SFH=1 Request_URL=0 having_IP_Address=0 147 ==> Result=-1 135    <conf:(0.92)> lift:(1.77) lev:(0.04) [58] conv:(5.44)
 7. URL_of_Anchor=-1 web_traffic=0 Result=1 148 ==> having_IP_Address=0 135    <conf:(0.91)> lift:(1.03) lev:(0) [3] conv:(1.21)
 8. Request_URL=1 URL_of_Anchor=1 age_of_domain=1 having_IP_Address=0 149 ==> Result=-1 135    <conf:(0.91)> lift:(1.75) lev:(0.04) [57] conv:(4.78)
 9. URL_Length=-1 having_IP_Address=0 Result=-1 151 ==> SFH=1 135    <conf:(0.89)> lift:(1.58) lev:(0.04) [49] conv:(3.85)
10. popUpWidnow=0 web_traffic=0 age_of_domain=1 151 ==> having_IP_Address=0 135    <conf:(0.89)> lift:(1.01) lev:(0) [1] conv:(1.02)
```

c)

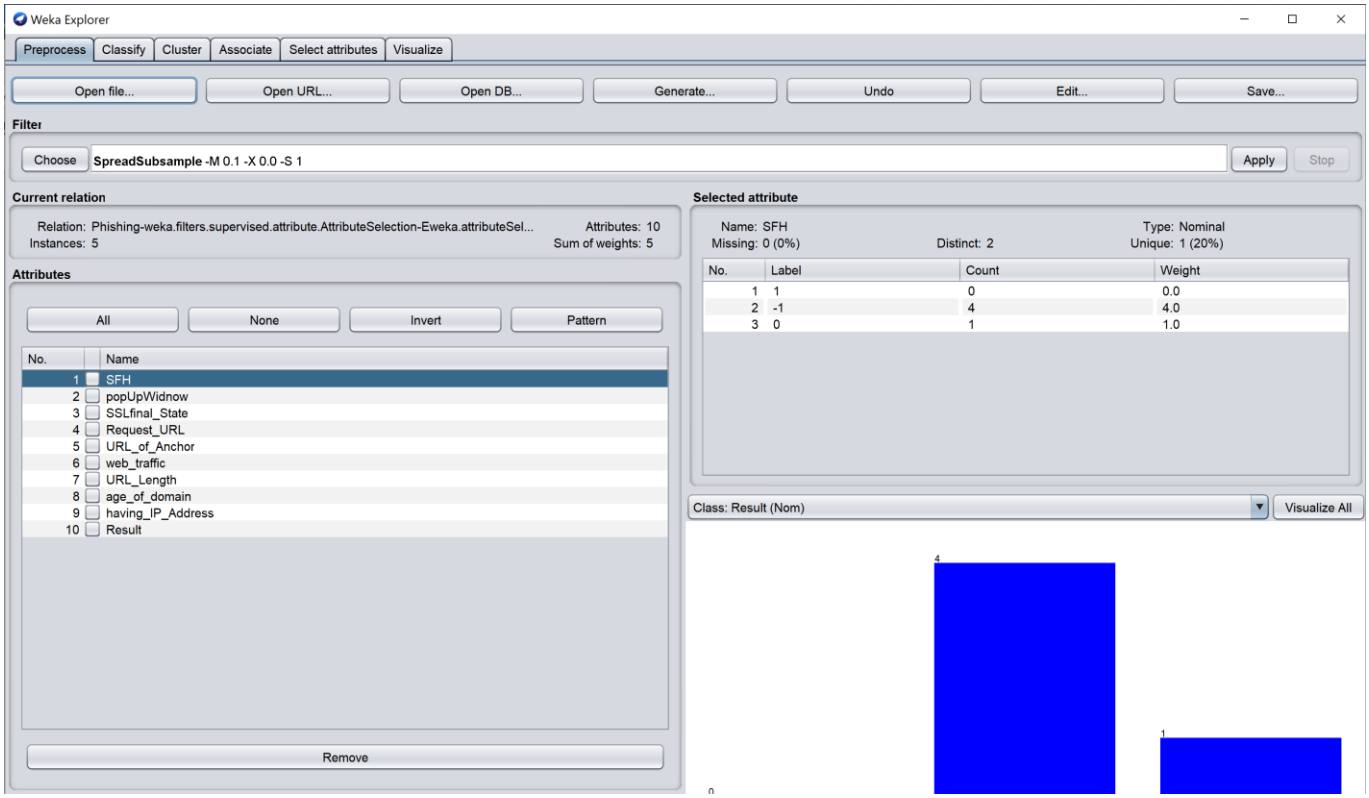
```
Apriori
=======

Minimum support: 0.5 (677 instances)
Minimum metric <confidence>: 0.8
Number of cycles performed: 10

Generated sets of large itemsets:

Size of set of large itemsets L(1): 5

Size of set of large itemsets L(2): 1

Best rules found:

 1. age_of_domain=1 825 ==> having_IP_Address=0 722    <conf:(0.88)> lift:(0.99) lev:(-0.01) [-8] conv:(0.91)
```

d)

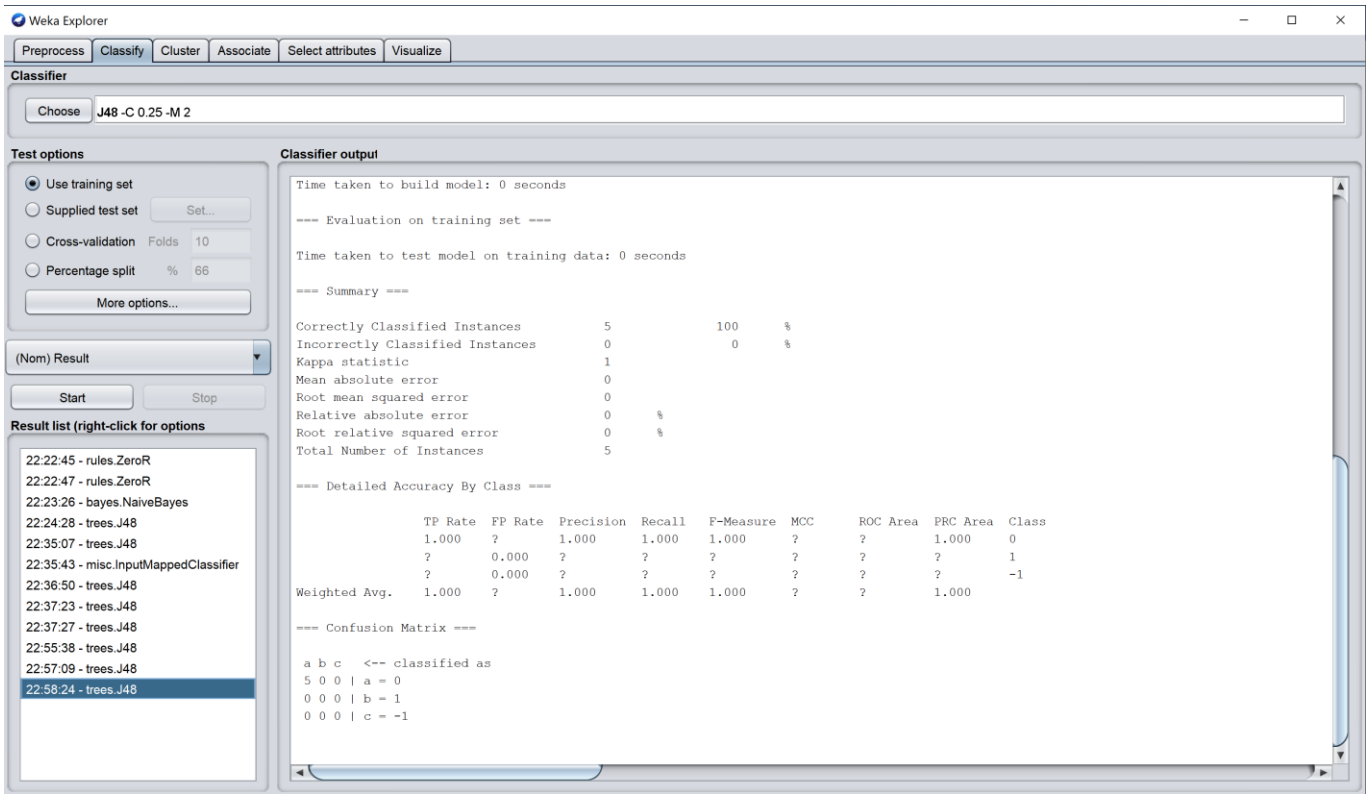


Clearly, the data is indeed unbalanced.

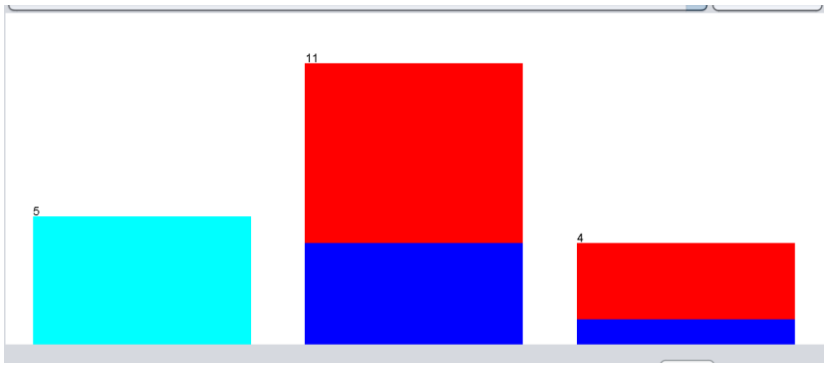
e)

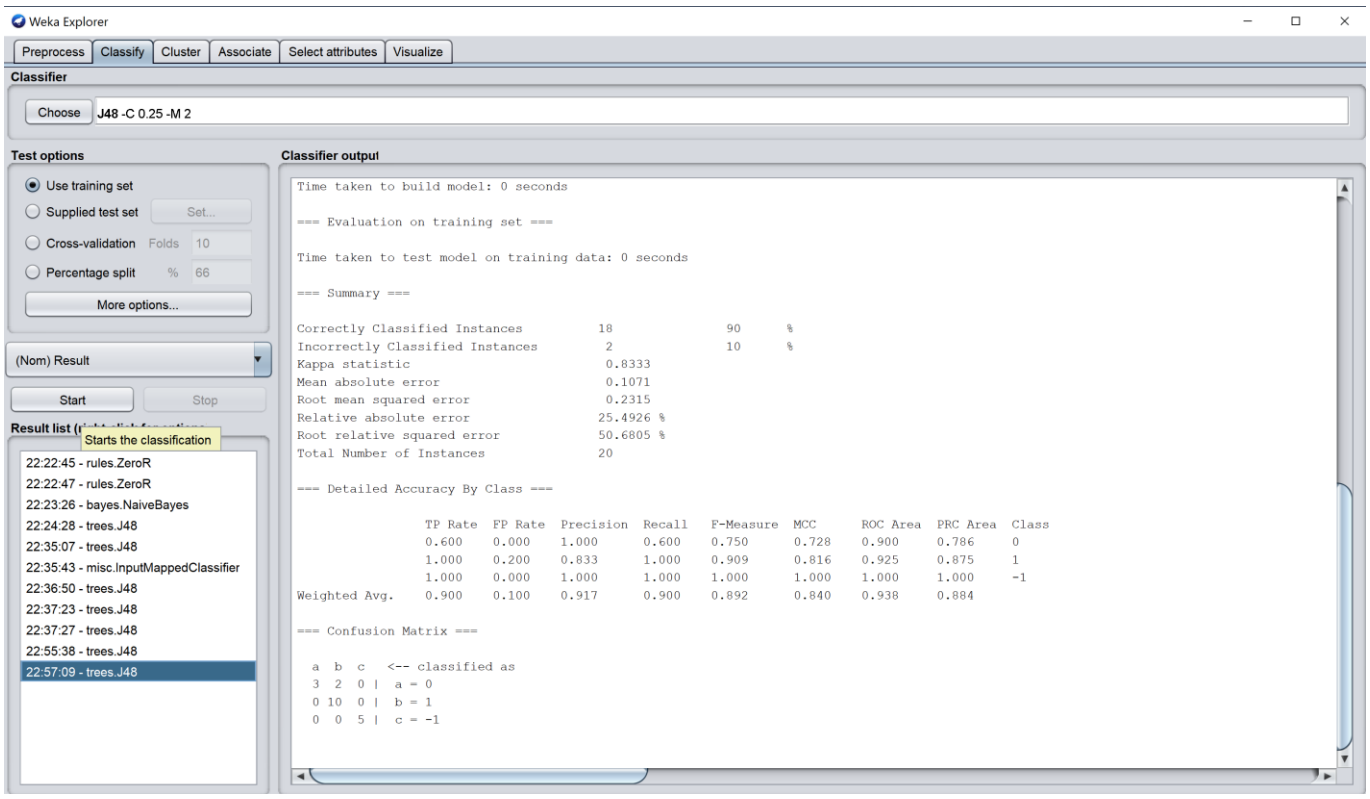

Weka Explorer
Preprocess
Classify
Cluster
Associate
Select attributes
Visualize
Open file...
Open URL...
Open DB...
Generate...
Undo
Edit...
Save...
Filter
Choose
SpreadSubsample -M 0.1 -X 0.0 -S 1
Apply
Stop
Current relation
Relation: Phishing-weka.filters.supervised.attribute.AttributeSelection-Eweka.attributeSel...
Attributes: 10
Instances: 5
Sum of weights: 5
Attributes
All
None
Invert
Pattern
No. Name
1 SFH
2 popUpWidnow
3 SSLfinal_State
4 Request_URL
5 URL_of_Anchor
6 web_traffic
7 URL_Length
8 age_of_domain
9 having_IP_Address
10 Result
Remove
Selected attribute
Name: SFH
Missing: 0 (0%)
Distinct: 2
Type: Nominal
Unique: 1 (20%)
No. Label Count Weight
1 1 0 0.0
2 -1 4 4.0
3 0 1 1.0
Class: Result (Nom)
Visualize All
4
1
0

f)


Weka Explorer
Preprocess
Classify
Cluster
Associate
Select attributes
Visualize
Classifier
Choose
J48 -C 0.25 -M 2
Test options
Use training set
Supplied test set
Set...
Cross-validation
Folds
10
Percentage split
%
66
More options...
(Nom) Result
Start
Stop
Result list (right-click for options
22:22:45 - rules.ZeroR
22:22:47 - rules.ZeroR
22:23:26 - bayes.NaiveBayes
22:24:28 - trees.J48
22:35:07 - trees.J48
22:35:43 - misc.InputMappedClassifier
22:36:50 - trees.J48
22:37:23 - trees.J48
22:37:27 - trees.J48
22:55:38 - trees.J48
22:57:09 - trees.J48
22:58:24 - trees.J48
Classifier output
Time taken to build model: 0 seconds
=== Evaluation on training set ===
Time taken to test model on training data: 0 seconds
=== Summary ===
Correctly Classified Instances 5 100 %
Incorrectly Classified Instances 0 0 %
Kappa statistic 1
Mean absolute error 0
Root mean squared error 0
Relative absolute error 0 %
Root relative squared error 0 %
Total Number of Instances 5
=== Detailed Accuracy By Class ===
TP Rate FP Rate Precision Recall F-Measure MCC ROC Area PRC Area Class
1.000 ? 1.000 1.000 1.000 ? ? 1.000 0
? 0.000 ? ? ? ? ? ? 1
? 0.000 ? ? ? ? ? ? -1
Weighted Avg. 1.000 ? 1.000 1.000 1.000 ? ? 1.000
=== Confusion Matrix ===
a b c <-- classified as
5 0 0 | a = 0
0 0 0 | b = 1
0 0 0 | c = -1

g) For Balanced Set ->

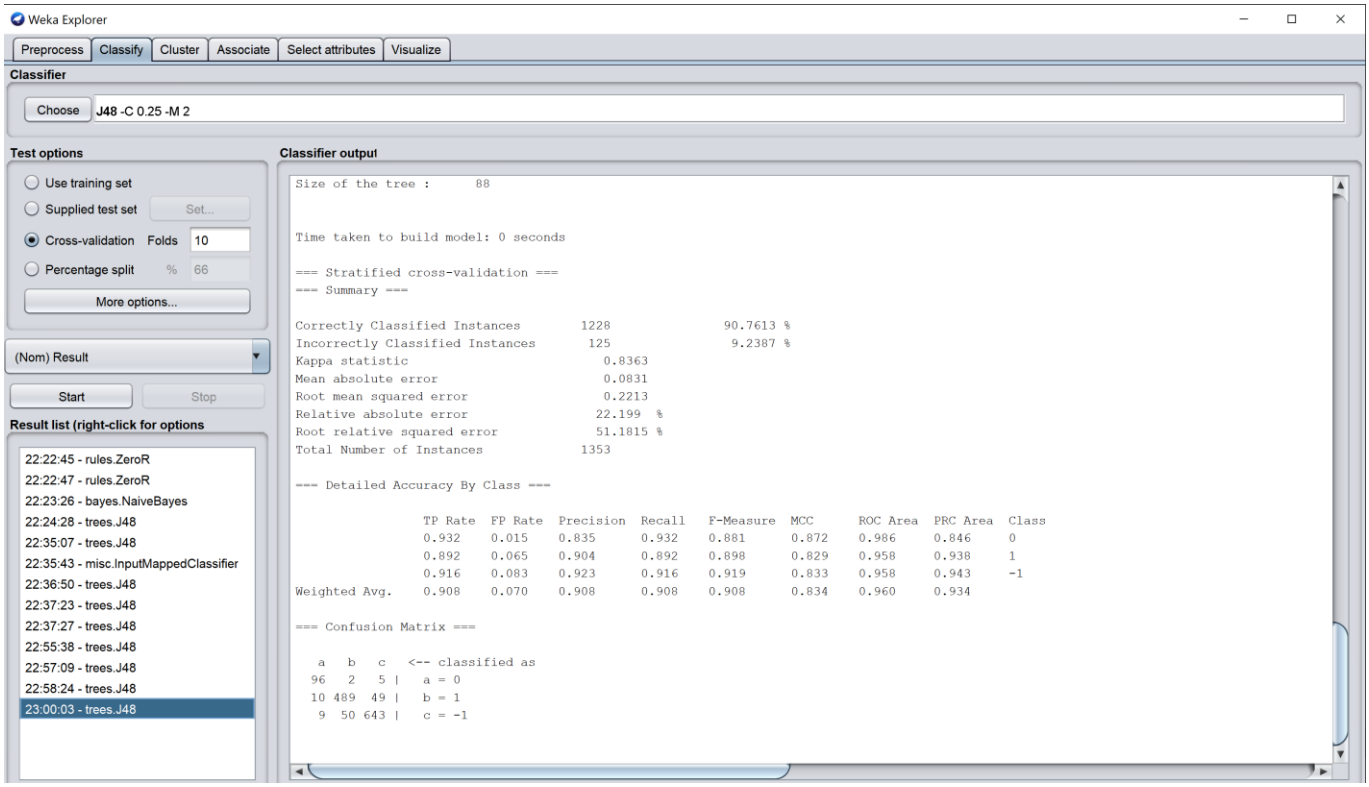




5. (a) Run the decision tree classifier with 10 fold cross validation (b) Record the average results (c) If you did 10 fold before, do 5 fold cross validation (d) Compare results and model sizes (RY O) (e) Run K-
means clustering with K=2 or K=3 (same as number of classes) (f) Can you measure the purity of your clusters? (g) Can you do anomaly detection in Weka? If so, how?

<u>Answer:</u>

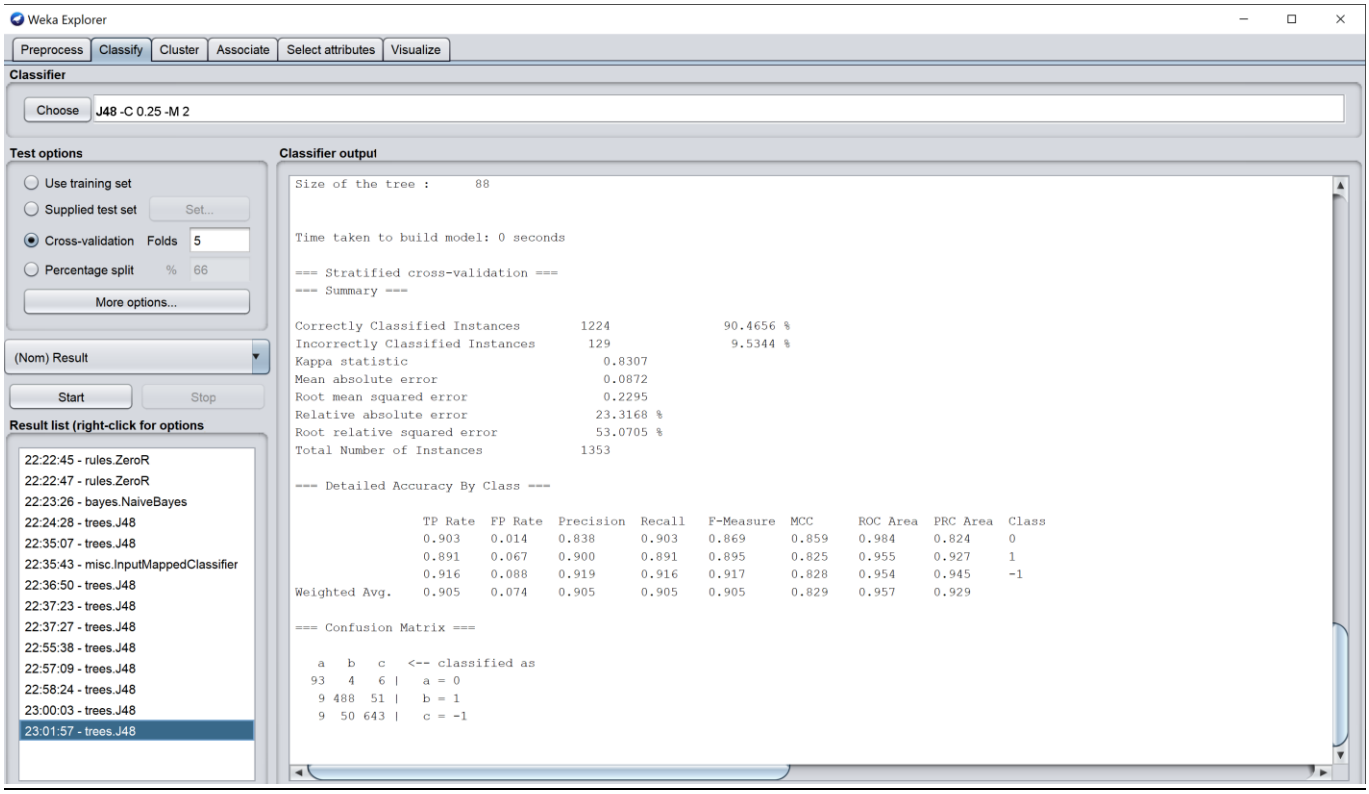
<u>a)</u>



b)

```
=== Detailed Accuracy By Class ===

                 TP Rate   FP Rate   Precision   Recall   F-Measure   MCC      ROC Area   PRC Area   Class
                 0.932     0.015     0.835       0.932    0.881       0.872    0.986      0.846      0
                 0.892     0.065     0.904       0.892    0.898       0.829    0.958      0.938      1
                 0.916     0.083     0.923       0.916    0.919       0.833    0.958      0.943      -1
Weighted Avg.    0.908     0.070     0.908       0.908    0.908       0.834    0.960      0.934

=== Confusion Matrix ===
```

c)



d)

```
=== Detailed Accuracy By Class ===

                 TP Rate  FP Rate  Precision  Recall   F-Measure  MCC      ROC Area  PRC Area  Class
                 0.903    0.014    0.838      0.903    0.869      0.859    0.984     0.824     0
                 0.891    0.067    0.900      0.891    0.895      0.825    0.955     0.927     1
                 0.916    0.088    0.919      0.916    0.917      0.828    0.954     0.945     -1
Weighted Avg.    0.905    0.074    0.905      0.905    0.905      0.829    0.957     0.929
```

e) k=3

```
kMeans
======

Number of iterations: 4
Within cluster sum of squared errors: 4383.0

Initial starting points (random):

Cluster 0: 1,0,1,1,1,-1,0,1,0,-1
Cluster 1: -1,1,1,-1,1,-1,0,1,0,0
Cluster 2: 0,-1,0,0,1,1,0,-1,0,1

Missing values globally replaced with mean/mode

Final cluster centroids:
                                 Cluster#
Attribute            Full Data          0           1           2
                      (1353.0)     (725.0)     (355.0)     (273.0)
==============================================================
SFH                         1           1          -1          -1
popUpWidnow                 0           0          -1          -1
SSLfinal_State              1           1           1          -1
Request_URL                -1           1          -1           0
URL_of_Anchor              -1           1          -1          -1
web_traffic                 0          -1           0           1
URL_Length                  0           0          -1           0
age_of_domain               1           1           1          -1
having_IP_Address           0           0           0           0
Result                     -1          -1           1           1
```

```
Clustered Instances

0       725 ( 54%)
1       355 ( 26%)
2       273 ( 20%)
```

f)

```
Time taken to build model (full training data) : 0.01 seconds

=== Model and evaluation on training set ===

Clustered Instances

0       713 ( 53%)
1       368 ( 27%)
2       272 ( 20%)


Class attribute: Result
Classes to Clusters:

   0   1   2  <-- assigned to cluster
  38  53  12 | 0
 100 252 196 | 1
 575  63  64 | -1

Cluster 0 <-- -1
Cluster 1 <-- 1
Cluster 2 <-- 0

Incorrectly clustered instances :       514.0    37.9897 %
```

PURITY ->

Cluster 0: 38/(38+100+575) = 38/713 = **0.05**

Cluster 1: 252/(53+252+63) = 252/368 = **0.68**

Cluster 2 = 64/(12+196+64) = 64/272 = **0.24**

g) **YES**

Weka has a Libsvm package which includes the one-class Support Vector Machine (SVM) classifier which is quite widely used for outlier detection problems.

-- Schölkopf, B., J. C. Platt, et al. (2001). "Estimating the Support of a High-Dimensional Distribution." Neural Computation 13(7): 1443-1471.